

PRAKTIKUM SISTEM OPERASI

MODUL 11

Memory Xinu



Oleh:

Jauhar Fajar Zuhair

2311104072

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK

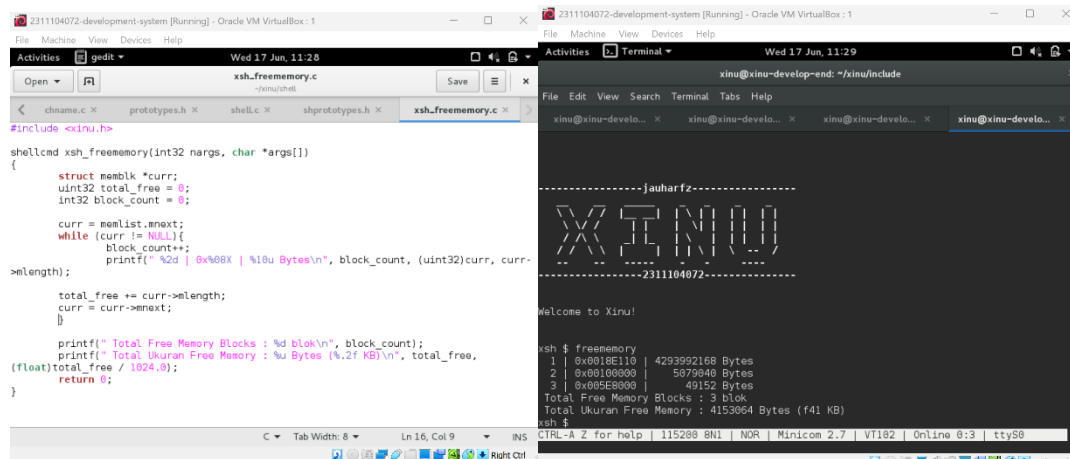
DIREKTORAT KAMPUS PURWOKERTO

UNIVERSITAS TELKOM

2026

JURNAL

1. Soal 1 a&b



```
#include <xinu.h>

shellcmd xsh_freememory(int32 nargs, char *args[])
{
    struct memblk *curr;
    uint32 total_free = 0;
    int32 block_count = 0;

    curr = memlist.next;
    while (curr != NULL) {
        block_count++;
        printf("%2d | %08X | %10u Bytes\n", block_count, (uint32)curr, curr->length);
        total_free += curr->length;
        curr = curr->next;
    }

    printf("Total Free Memory Blocks : %d blok\n", block_count);
    printf("Total Ukuran Free Memory : %u Bytes (%.2f KB)\n", total_free,
           (float)total_free / 1024.0);
    return 0;
}
```

```
-----jauharfz-----
XINU
-----2311104072-----

Welcome to Xinu!

xsh $ freememory
1 | 0x0018E110 | 4293992168 Bytes
2 | 0x00100000 | 5079940 Bytes
3 | 0x005E8000 | 49152 Bytes
Total Free Memory Blocks : 3 blok
Total Ukuran Free Memory : 4153064 Bytes (f41 KB)
xsh $
```

2. Soal 2

- Mengapa Xinu memisahkan data segment dan BSS segment? Untuk menghemat ukuran file *image* OS. Data segment menyimpan variabel global yang memiliki nilai awal (perlu disimpan di disk), sedangkan BSS segment menyimpan variabel global yang belum diinisiasi (hanya perlu dialokasikan di RAM saat *booting* tanpa memperbesar ukuran file *image*)
- Bagaimana alokasi dan dealokasi memori selama eksekusi memengaruhi ukuran free space? Alokasi memori (pembuatan proses/penggunaan `getmem`) akan mengurangi ukuran *free space*, sedangkan dealokasi memori (terminasi proses/penggunaan `freemem`) akan menambah kembali ukuran *free space* secara dinamis
- Mengapa penggunaan heap lebih berpotensi menimbulkan masalah dibandingkan stack? Karena alokasi *stack* otomatis dihapus saat fungsi/proses selesai (LIFO), sedangkan *heap* harus di-dealokasi secara manual. Jika lupa, akan menyebabkan kebocoran memori (*memory leak*)
- Mengapa Xinu menggunakan struktur linked list untuk menyimpan free block? Karena ukuran dan lokasi blok memori kosong sangat dinamis (tidak berurutan). *Linked list* (`memlist`) memudahkan Xinu untuk menyisipkan, menghapus, dan menggabungkan blok memori yang bebas secara fleksibel.

e. Apa tantangan utama dalam penggunaan heap di Xinu? Tantangan utamanya adalah terjadinya fragmentasi memori (memori terpecah menjadi blok-blok kecil yang tidak berurutan) dan risiko memory leak karena tidak adanya fitur *Garbage Collector* otomatis di Xinu.